

TWO POINTERS

A complete learning chapter for understanding the pattern, recognizing it in interviews, deriving the solution, coding it in Java, and practicing it systematically.

THE BIG IDEA

Two Pointers uses two indices to shrink a search space or maintain a useful relationship between elements. The real skill is not memorizing pointer code — it is proving why a pointer can safely move.

Goal: turn repeated scanning into controlled movement, often reducing $O(n^2)$ work to $O(n)$.

Chapter map

- Understand the intuition and the brute-force → optimized journey.
- Master opposite-end, slow/fast, partition, merge, and linked-list pointer patterns.
- Learn reusable Java syntax and implementation templates.
- Study representative problems from basic to advanced.
- Learn invariants, edge cases, debugging, complexity, and interview explanation.
- Know when Two Pointers is the wrong tool and when Sliding Window is the better abstraction.

01 · What Is Two Pointers?

Two Pointers is a family of techniques where two indices move through an input according to a rule. Their movement is designed so that each step removes impossible choices, maintains a valid region, or constructs the answer.

Brute force first

```
arr = [1, 2, 4, 6, 8, 9]
target = 10

# Brute force idea
for i:
    for j > i:
        if arr[i] + arr[j] == target:
            answer
```

The problem: many pairs are checked even after the sorted order has already told us that some pairs are impossible.

Optimization insight

```
1 2 4 6 8 9
  ↑ ↑
  L R
```

If $\text{arr}[L] + \text{arr}[R]$ is too small, moving R left only makes the sum smaller. Therefore R cannot be the fix — move L.

The important mental shift

Do not ask only “What code should I write?” Ask “What information lets me eliminate a whole set of possibilities?” That elimination rule is what makes Two Pointers efficient.

Three questions before coding

- What does each pointer represent? Candidate, boundary, scanner, writer, or position in another array?
- What makes a pointer move? The movement must be justified by the problem's ordering or invariant.
- What remains guaranteed? State the invariant in one sentence.

02 · Pattern Recognition

The interview skill is recognizing the structure quickly.

Signal in the problem	Likely pattern
Sorted array + pair relationship	Opposite ends
Compare characters/elements from both ends	Opposite ends
Keep only valid elements in-place	Slow + fast
Move all values satisfying a condition together	Partition
Two sorted arrays / sequences	Merge pointers
Linked list cycle / middle / positional gap	Fast + slow
Triplets such as 3Sum	Fix one + opposite-end pointers

Recognition shortcut

- **SORTED + PAIR?** → start by testing opposite ends.
- **FILTER / COMPACT?** → think reader + writer (fast + slow).
- **TWO SORTED INPUTS?** → merge pointers.
- **REARRANGE AROUND A CONDITION?** → partition pointers.
- **LINKED LIST + SPEED / CYCLE?** → fast + slow.
- **TRIPLETS?** → sort, fix one index, solve the remaining pair with two pointers.

Important warning

Two Pointers is not “two variables named left and right.” It is a correctness strategy. If you cannot explain why a pointer move eliminates impossible states or preserves the invariant, do not code yet.

03 · Pattern A — Opposite Ends

One pointer begins at the left boundary and the other at the right boundary. The region between them is the remaining search space.

Template

```
int left = 0;
int right = arr.length - 1;

while (left < right) {
    // inspect arr[left] and arr[right]

    if (/* left side should move */) {
        left++;
    } else {
        right--;
    }
}
```

Why the movement works

Suppose the array is sorted.

If the current sum is < target → move left rightward.

If the current sum is > target → move right leftward.

If equal → answer found.

Invariant

At every iteration, every pair that has already been ruled out remains impossible; the answer, if it exists, is still inside the remaining pointer region.

Canonical problem: Two Sum II

```
Arrays.sort(arr); // only if the problem permits / needs sorting
```

```
int left = 0, right = arr.length - 1;
```

```
while (left < right) {
    int sum = arr[left] + arr[right];

    if (sum == target) {
        return new int[]{left, right};
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
return new int[]{-1, -1};
```

Complexity: $O(n)$ after the array is already sorted. If you sort first: $O(n \log n) + O(n)$.

04 · Dry Runs: Learn the Movement

A good dry run should explain not just what happens, but why the pointer moves.

Example 1 — sum too small

```
1 2 4 6 8 9
  ↑ ↑
  L R
1 + 9 = 10 → found
```

Example 2 — sum too small

```
1 2 4 6 8 9
  ↑ ↑
  L R
1 + 8 = 9 < 10 → move L
2 4 6 8 9
  ↑ ↑
  L R
2 + 8 = 10 → found
```

Example 3 — sum too large

```
2 5 7 11 15
  ↑ ↑
  L R
2 + 15 = 17 > 12 → move R
2 5 7 11 15
  ↑ ↑
  L R
2 + 11 = 13 > 12 → move R again
```

Rule to internalize

Every pointer move must make a permanent statement: “This eliminated position cannot be part of a valid answer.”

05 · Pattern B — Same Direction / Slow & Fast

Here both pointers move left → right, but they have different jobs. Fast explores; slow marks where the next valid answer belongs.

Template

```
int slow = 0;

for (int fast = 0; fast < n; fast++) {
    if (isValid(arr[fast])) {
        arr[slow] = arr[fast];
        slow++;
    }
}
```

Core invariant

The prefix `0..slow-1` is already the correct compacted answer. `fast` points at the next element to inspect.

Problem: Remove Duplicates from Sorted Array

```
int slow = 0;

for (int fast = 1; fast < nums.length; fast++) {
    if (nums[fast] != nums[slow]) {
        nums[++slow] = nums[fast];
    }
}

return slow + 1;
```

Problem: Move Zeroes

```
int slow = 0;

for (int fast = 0; fast < nums.length; fast++) {
    if (nums[fast] != 0) {
        int temp = nums[slow];
        nums[slow] = nums[fast];
        nums[fast] = temp;
        slow++;
    }
}
```

Think: `fast` = reader • `slow` = writer / boundary

06 · Pattern C — Partition Pointers

Partitioning divides the array into meaningful regions. The pointers maintain those regions while an unknown region shrinks.

Two-way partition

```
int left = 0;
int right = nums.length - 1;

while (left <= right) {
    while (left <= right && nums[left] < pivot) left++;
    while (left <= right && nums[right] >= pivot) right--;

    if (left < right) {
        swap(nums, left, right);
        left++;
        right--;
    }
}
```

Three-way partition — Dutch National Flag

```
int low = 0, mid = 0, high = nums.length - 1;

while (mid <= high) {
    if (nums[mid] == 0) {
        swap(nums, low++, mid++);
    } else if (nums[mid] == 1) {
        mid++;
    } else {
        swap(nums, mid, high--);
    }
}
```

Invariant / regions

```
[ 0 ... low-1 ] = 0
[ low ... mid-1 ] = 1
[ mid ... high ] = unknown
[ high+1 ... n-1 ] = 2
```

Why this is powerful

Instead of sorting every element with a general-purpose sort, the algorithm exploits the small fixed set of categories and maintains the answer regions directly.

07 · Pattern D — Merge Two Sorted Sequences

If two sequences are sorted, compare their current elements. Once the smaller element is chosen, it never needs to be considered again.

Merge template

```
int i = 0, j = 0, k = 0;

while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) {
        result[k++] = a[i++];
    } else {
        result[k++] = b[j++];
    }
}

while (i < a.length) result[k++] = a[i++];
while (j < b.length) result[k++] = b[j++];
```

Intersection template

```
int i = 0, j = 0;

while (i < a.length && j < b.length) {
    if (a[i] == b[j]) {
        // common value
        i++;
        j++;
    } else if (a[i] < b[j]) {
        i++;
    } else {
        j++;
    }
}
```

Movement rule

If $a[i] < b[j]$, $a[i]$ cannot match $b[j]$ or anything before it — so advance i . Symmetrically, if $b[j]$ is smaller, advance j .

Complexity: $O(n + m)$ time because each pointer advances at most the length of its sequence.

08 · Pattern E — Fast & Slow in Linked Lists

Two Pointers becomes especially useful when the structure does not support random access. Different pointer speeds create useful positional relationships.

Middle of linked list

```
ListNode slow = head;
ListNode fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}

return slow;
```

Cycle detection — Floyd's algorithm

```
ListNode slow = head;
ListNode fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;

    if (slow == fast) {
        return true;
    }
}

return false;
```

Why different speeds help

If a cycle exists, the faster pointer eventually catches the slower pointer inside the cycle. If there is no cycle, fast reaches null.

Cycle start idea

```
// After detecting a meeting point:
ListNode p1 = head;
ListNode p2 = meeting;

while (p1 != p2) {
    p1 = p1.next;
    p2 = p2.next;
}

return p1; // cycle entry
```

Other linked-list problems using the same family

- Remove Nth Node From End
- Palindrome Linked List
- Find Middle Node
- Detect Cycle
- Find Cycle Entry

09 · Java Toolkit for Two Pointers

These are the small pieces of syntax that repeatedly appear in pointer problems.

Arrays

```
int n = arr.length;
int x = arr[i];
arr[i] = value;
```

```
Arrays.sort(arr);
```

Characters

```
Character.isLetterOrDigit(c)
Character.toLowerCase(c)
Character.isWhitespace(c)
```

Lists

```
List<Integer> list = new ArrayList<>();
list.add(x);
int x = list.get(i);
Collections.sort(list);
```

Swap

```
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
```

Swap helper

```
static void swap(int[] a, int i, int j) {
    int temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
```

String

```
int n = s.length();
char c = s.charAt(i);

StringBuilder sb = new StringBuilder();
```

Loop shapes worth memorizing

```
// Opposite ends
while (left < right) {
    ...
}

// Reader / writer
int slow = 0;
for (int fast = 0; fast < n; fast++) {
    if (valid(arr[fast])) {
        arr[slow++] = arr[fast];
    }
}

// Two sorted arrays
while (i < n && j < m) {
    ...
}
```

Java trap: integer overflow

```
long sum = (long) nums[left] + nums[right];
```

Use this when constraints can make the sum exceed int range.

10 · Problem Roadmap

Do not jump randomly between problems. Learn the pattern in increasing difficulty.

▣ LEVEL 1 · FOUNDATION

- Reverse String
- Reverse Array
- Valid Palindrome
- Two Sum II — Input Array Is Sorted

▣ LEVEL 2 · PATTERN BUILDING

- Remove Duplicates from Sorted Array
- Move Zeroes
- Squares of a Sorted Array
- Merge Sorted Array
- Intersection of Two Arrays

▣ LEVEL 3 · INTERVIEW

- Container With Most Water
- 3Sum
- Sort Colors
- Backspace String Compare
- Boats to Save People

▣ LEVEL 4 · ADVANCED

- Trapping Rain Water
- 4Sum
- Linked List Cycle
- Linked List Cycle II
- Remove Nth Node From End
- Palindrome Linked List

Problem → pattern map

Problem	Pattern	Main lesson
Valid Palindrome	Opposite ends	Compare / skip from both sides
Two Sum II	Opposite ends	Monotonic pointer movement
Remove Duplicates	Slow/Fast	Reader + writer invariant
Move Zeroes	Slow/Fast	In-place rearrangement
Container	Opposite ends	Greedy pointer choice
3Sum	Fix + Two Pointers	Combine patterns + duplicates
Sort Colors	Partition	Maintain multiple regions
Merge Sorted Array	Merge	Consume the smaller side
Cycle Detection	Fast/Slow	Speed creates positional information

11 · Deep Dive — 3Sum

3Sum is important because it demonstrates how Two Pointers can be nested inside another loop.

Derivation

- Sort the array so the remaining pair has an ordering we can exploit.
- Fix `nums[i]` as the first number.
- Search for a pair in `i+1..n-1` whose sum equals `-nums[i]`.
- Use left/right exactly like Two Sum II.
- Skip duplicates at `i` and after finding a triplet.

```
Arrays.sort(nums);

for (int i = 0; i < nums.length - 2; i++) {

    if (i > 0 && nums[i] == nums[i - 1]) continue;

    int left = i + 1;
    int right = nums.length - 1;

    while (left < right) {
        int sum = nums[i] + nums[left] + nums[right];

        if (sum == 0) {
            // record triplet

            left++;
            right--;

            while (left < right && nums[left] == nums[left - 1])
                left++;
            while (left < right && nums[right] == nums[right + 1])
                right--;

        } else if (sum < 0) {
            left++;
        } else {
            right--;
        }
    }
}
```

Complexity

Sorting: $O(n \log n)$. For each fixed `i`, the two-pointer scan is $O(n)$. Overall: $O(n^2)$.

Common 3Sum mistakes

- Forgetting to sort.
- Not skipping duplicate `i` values.
- Skipping duplicates before recording the current valid triplet.
- Using left/right outside the correct range.
- Overflow when adding large integers — consider long.

12 · Deep Dive — Container With Most Water

This problem teaches a different kind of pointer movement: a greedy decision based on the limiting side.

```
int left = 0;
int right = height.length - 1;
int best = 0;

while (left < right) {
    int width = right - left;
    int h = Math.min(height[left], height[right]);

    best = Math.max(best, width * h);

    if (height[left] < height[right]) {
        left++;
    } else {
        right--;
    }
}
```

Why move the shorter side?

Area = width × min(leftHeight, rightHeight). Width always decreases when either pointer moves. If the shorter side stays, the limiting height cannot improve enough to justify keeping it while width shrinks. Therefore, move the shorter side.

Interview lesson

This is a classic example where the correct pointer movement is a greedy proof, not an arbitrary coding trick.

13 · Edge Cases & Debugging

Before submitting a pointer solution, deliberately test the boundaries.

Array checklist

- Empty array
- One element
- Two elements
- All values equal
- Already sorted
- Reverse sorted
- Many duplicates
- No valid answer
- Multiple valid answers
- Negative numbers
- Very large values / overflow
- `left == right` boundary

String checklist

- Empty string
- One character
- Spaces
- Upper/lowercase
- Special characters
- Only invalid/skipable characters

Common mistakes

Moving the wrong pointer

The movement must eliminate impossible states.

Using the wrong loop condition

For pairs, usually `left < right`; for inclusive partitioning, often `left <= right`.

Forgetting that sorting changes the array

Check whether mutation/order/index identity matters.

Duplicate handling in 3Sum

Skip duplicates at the fixed index and after a found triplet.

Off-by-one errors

Write pointer ranges and region meanings before coding.

Accidental $O(n^2)$

Do not reset a pointer inside a loop if the intended pattern is linear.

Using Two Pointers without an ordering argument

If you cannot prove monotonic elimination, another technique may be needed.

Debugging question

At every iteration ask: $L = ?$ • $R = ?$ • $values = ?$ • $invariant = ?$ • **why is this movement safe?**

14 · Two Pointers vs Sliding Window

Sliding Window is often implemented using two pointers, which is why the two ideas are easy to confuse.

	Two Pointers	Sliding Window
Typical shape	L and R represent search positions / candidates	[L ... R] represents a contiguous window
Main goal	Eliminate candidates, compare, partition, merge, or build	Maintain a property of a contiguous range
Typical signal	Pair, ends, sorted inputs, in-place rearrangement	Longest/shortest subarray or substring
Examples	Two Sum II, 3Sum, Container	Longest substring, Minimum Window, max sum window

Rule of thumb: if the problem asks about a contiguous range and you maintain information about that range as it expands/shrinks, think Sliding Window. If pointers are independently moving to eliminate candidates or compare positions, think Two Pointers.

Don't force Two Pointers

- Unsorted pair lookup with no useful ordering → HashMap may be $O(n)$.
- Binary decision over a sorted search space → Binary Search.
- Frequency/counting requirement → HashMap / frequency array may be better.
- Contiguous range property → Sliding Window is usually clearer.
- Prefix relationships → Prefix Sum may be the natural solution.

15 · Complexity Patterns

Pattern	Typical time	Extra space	Why
Opposite ends	$O(n)$	$O(1)$	Each pointer moves inward at most n times
Slow/Fast	$O(n)$	$O(1)$	Fast scans once; slow never moves backward
Partition	$O(n)$	$O(1)$	Unknown region shrinks every step
Merge	$O(n+m)$	$O(1)$ or $O(n)$	Each input pointer advances monotonically
3Sum	$O(n^2)$	$O(1)$ extra*	Outer loop $\times$ linear two-pointer scan
Sort + pointers	$O(n \log n) + \text{scan}$	Depends	Sorting creates the ordering used by the pointers

* Output storage for problems such as 3Sum can make total space larger; “extra space” refers to auxiliary algorithmic space.

Why Two Pointers becomes $O(n)$

The key property is monotonic movement. If left and right each move only forward/inward and never reset, the total number of pointer moves is $O(n)$, not $O(n^2)$.

16 · How to Explain a Two-Pointer Solution in an Interview

A strong explanation should connect the data property → pointer setup → movement proof → invariant → complexity.

Use this five-step script

- 1. Observation: "The array is sorted, so the relationship between values is monotonic."
- 2. Setup: "I place one pointer at each end."
- 3. Movement: "If the sum is too small, increasing left is the only move that can increase it; if too large, decreasing right is the only move that can decrease it."
- 4. Correctness: "Each move eliminates positions that cannot produce the answer, so we never discard a valid solution."
- 5. Complexity: "Each pointer moves at most n times, so time is $O(n)$ and extra space is $O(1)$."

Example answer — Two Sum II

"Because the array is sorted, I can avoid checking every pair. I start one pointer at the smallest value and one at the largest. If their sum is below the target, I move the left pointer because moving the right pointer left would only make the sum smaller. If the sum is above the target, I move the right pointer. Each move permanently eliminates an impossible set of pairs. Since each pointer moves at most n positions, the time complexity is $O(n)$ and extra space is $O(1)$."

If asked for proof

Point to the exact monotonic property that makes a move safe. That is the heart of the algorithm.

17 · Two-Pointer Decision Tree

Question	If YES	If NO
Is input sorted or can it safely be sorted?	Try opposite ends / merge	Continue
Is it a pair/triplet relationship?	Try Two Pointers	Continue
Do you need to compact/filter in-place?	Slow + fast	Continue
Do you need multiple regions / rearrangement?	Partition	Continue
Are there two sorted inputs?	Merge pointers	Continue
Is it a linked list with speed/position/cycle?	Fast + slow	Consider another pattern
Is the target a contiguous range property?	Consider Sliding Window	Consider other patterns

Before coding, write these four lines on paper

1. Pointer meaning: _____
2. Pointer initialization: _____
3. Movement rule: _____
4. Invariant: _____

18 · Final Cheat Sheet

OPPOSITE ENDS

left = 0 • right = n-1 • compare ends • move the side justified by ordering

SLOW / FAST

fast = reader • slow = writer/boundary • preserve the correct prefix

PARTITION

Maintain regions • inspect unknown region • swap / advance until unknown region disappears

MERGE

Compare current values • consume the smaller / selected side • never move backward

LINKED LIST FAST / SLOW

slow += 1 • fast += 2 • useful for middle, cycle, positional gaps

3SUM

Sort → fix i → left/right → move by sum → skip duplicates

THE FIVE QUESTIONS

- What do my pointers represent?
- What is the invariant?
- Why is this pointer safe to move?
- Can each pointer move only once / monotonically?
- What are the edge cases and complexity?

Practice progression

Reverse String → Valid Palindrome → Two Sum II → Remove Duplicates → Move Zeroes → Merge → Container → Sort Colors → 3Sum → Linked List Cycle → Trapping Rain Water

FINAL RULE

Do not memorize “left++” or “right--”. Memorize the reason that makes the movement safe.

End of chapter • Two Pointers